

# **PULSAR: Dynamic Time-Slice Scaling Via Q-Learning for Mixed Workload Embedded RTOS**

Braden Everson

Dwight and Dian Diercks School  
of Advanced Computing  
Milwaukee School of Engineering  
Milwaukee, WI 53202  
eversonb@msoe.edu

James Lembke

Dwight and Dian Diercks School  
of Advanced Computing  
Milwaukee School of Engineering  
Milwaukee, WI 53202  
lembke@msoe.edu

## **Abstract**

Standard scheduling in embedded Real-Time Operating Systems (RTOS) often fails to adapt to tasks that fluctuate between CPU-intensive and I/O-bound phases, resulting in increased system-wide latency. We present PULSAR (Personalized Utilization-based Learning for Scheduling and Allocating Runtime), a Q-learning-based framework that dynamically adjusts task time-slices based on discretized snapshots of task execution patterns. PULSAR learns to allocate larger time quanta during I/O-bound phases to expedite I/O entry, while minimizing context-switch frequency and maintaining fairness during CPU-intensive periods. By maintaining a unique Q-learning agent and quantum table for each task, PULSAR provides a personalized scheduling response tailored to individual task signatures.

We evaluate PULSAR using average system wait time across various workloads in both simulated and bare-metal environments. Results demonstrate that PULSAR reduces task wait times by an average of 5% compared to a fixed 10ms Round Robin scheduler. With a modest memory footprint of 580 bytes per task and constant-time evaluation, PULSAR enhances system responsiveness with minimal overhead. This paper explores the performance tradeoffs of PULSAR across heterogeneous workloads and its viability for resource-constrained real-time applications.

# 1 Introduction

The efficiency of embedded Real Time Operating Systems (RTOS) [1] is fundamentally tied to how scheduling decisions are made across the tasks that a system may manage. In many modern systems, workloads are rarely uniform. Tasks may constantly switch between phases of heavy computational work and needing to read large bursts of data from a peripheral or subsystem. Static schedulers often fail to understand and adapt to this dynamic behavior, and therefore might make the wrong choice at the cost of system efficiency. Traditional scheduling algorithms, such as Round Robin and Earliest Deadline First, typically rely on fixed time-slices (time quanta). While predictable, these static quanta often force a task to undergo an unnecessary context switch just milliseconds before it would have naturally entered an I/O wait state, leading to increased system-wide latency and wasted overhead.

The challenge in optimizing these intervals lies in the diversity of task signatures. A "one-size-fits-all" dynamic scheduler often fails to account for the unique behaviors and state transitions of individual tasks. While recent research has explored Deep Reinforcement Learning (DRL) for task scheduling, the high computational cost and memory requirements of neural networks often make them impractical for resource-constrained, bare-metal environments. There remains a need for a lightweight, adaptive mechanism that can learn and respond to task-specific behavior in real-time.

In this paper, we introduce PULSAR (Personalized Utilization-based Learning for Scheduling and Allocating Runtime). PULSAR uses a decentralized Q-learning framework where each task is assigned its own agent aiming to optimize that specific task's time quantum allocation. These agents learn to classify task behavior in discrete bins and adjust quantum allocations dynamically. By providing a larger quantum during I/O-heavy phases, PULSAR accelerates the transition to I/O waiting queues, freeing the CPU for other ready tasks and reducing the average system wait time while balancing the total fairness of the system as a whole.

## 2 Background

Real Time Operating Systems (RTOS) are a subset of traditional Operating Systems that focus on time constrained tasks. They are especially seen in the world of embedded systems where there is less computational power and higher urgency for system efficiency. RTOS primarily use one of two algorithms to choose which task is scheduled for the CPU next: Round Robin and Earliest Deadline First (EDF). EDF was first proposed by C. L. Liu and J. W. Layland in "Scheduling algorithms for multiprogramming in a hard-real-time environment," where they describe the benefits of scheduling the task that has the earliest deadline to meet first [3]. PULSAR will focus on a simpler, Round Robin approach in an RTOS with long living tasks that don't have immediate deadlines. Such a system is common in

embedded systems such as 3D Printers, Autonomous Vehicles, and IoT applications.

Another common practice in operating systems is to use priority queues to allow more important tasks to run first. PULSAR does not currently aim to use priority queues, as we focus solely on how quantum allocation can affect system fairness regardless of priority. With this system in place, PULSAR could easily be paired with an existing priority queuing mechanism to gain the benefit of both parts. Prior work has explored dynamic queuing [7], and will serve as a future comparison point to PULSAR.

The wait time of a given task can be measured as how much time it spends "wanting" to perform work. Essentially, this is only the time the task spends in the ready waiting queue while another task is currently running on the CPU. When a task has made an I/O call and is instead waiting for the response from that system, it is not wanting the CPU and therefore is not wasting time. Time quantum allocation can significantly affect how much time a task spends waiting, as a task that will enter an I/O wait in  $n$  ms will reach that point if given a quantum greater than or equal to this value. If the task is allocated less than  $n$  ms, it will be placed on the ready waiting queue again and have to wait until its turn again before then performing the I/O request, which it could have done previously if only given more time to do so. This is exactly what PULSAR aims to solve, dynamic quantum allocation to reach I/O waiting states more frequently.

## 2.1 Q-Learning

Q-Learning is a model-free reinforcement learning algorithm used to determine an optimal policy by maximizing cumulative rewards within a given environment [6]. As outlined by Watkins and Dayan (1992), the algorithm maintains Q-values representing the expected utility of taking specific actions in given states. Q-Learning is a good choice for implementing in an RTOS context switch due to its lower runtime overhead and memory footprint compared to other modeling approaches such as neural networks.

In the case of PULSAR, Q-Learning will be used to approximate the best quantum allocations for each task separately with respect to the current behavior the task is exhibiting.

## 3 Related Work

Sun et al. [4] explores the implementation of a Double Deep Q-Network (Double DQN) to optimize process scheduling in general-purpose operating systems. Unlike standard Q-Learning, which relies on discrete state-action tables, this work utilizes deep neural networks to handle high-dimensional state spaces, including complex metrics like multi-level queue depths, historical CPU burst patterns, and process priorities. However, the DQN net-

work layers expose too high of overhead to be practical in a bare-metal RTOS environment,

Li et al. [5] proposes a model-free reinforcement learning framework to optimize resource allocation in dynamic computer systems. The authors utilize a Q-learning algorithm to manage system variables including CPU utilization, memory usage, and task queue length. By defining the system state as a multidimensional vector and employing an  $\epsilon$ -greedy strategy, the agent learns to balance immediate performance gains against long-term resource stability. However, this work utilizes a global system-state approach. The approach presented by PULSAR provides a personalized “per-task” time slicing producing better time slicing in diverse processing workloads.

Zhong, W. [7] presents an optimization method for Multi-Level Feedback Queue (MFQ) scheduling by integrating a Deep Q-Network (DQN) with a dynamic time-slice adjustment strategy. The system uses a DQN to dynamically select the most suitable execution queue based on current queue lengths, rather than strictly adhering to fixed priority levels. However, the DQN utilizes 64 parameters with significant memory and computational costs when compared to the 580 overhead of PULSAR. Where Zhong, W’s proposed DQN uses a neural network with a heavier floating point operation cost [7], PULSAR’s evaluation only requires the resolution of a 5 floating point operations to derive the reward value needed to drive the rest of the agent’s decisions. Further, with 64 parameters of 32-bit floats, a DQN neural network of that complexity requires at least 2048 bytes to store parameters alone, compared to PULSAR’s 580 byte overhead.

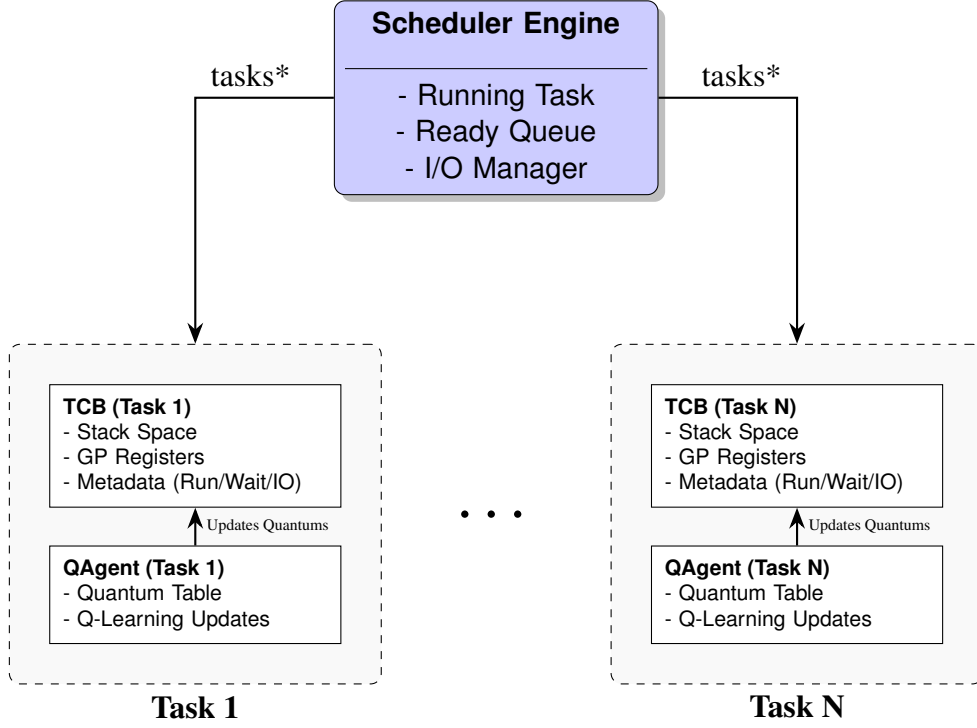
## 4 Methods

In this section, we will highlight the core architectures during the phases of development and testing on PULSAR. In early stages of rapidly prototyping the agent architecture, a simulated RTOS was used. From this, a true bare-metal RTOS was built that runs on an Stm32F446 microcontroller. With the RTOS created, trivial test workloads were developed to validate that the agent would minimize system wait times. Finally, a series of workloads with inspiration set in the 3D Printer theme of tasks were created to test how PULSAR adapts to realistic long running workloads.

### 4.1 Abstract Architecture

Both the simulated and real RTOS environments share a common architecture composed of the Scheduler Engine which manages one or more Task Control Blocks (TCBs). Each TCB has data on the current stack and GP register values, alongside tracked metadata on task run and wait times. Paired with each TCB is a corresponding unique QAgent that is responsible for maintaining a bucketized table of quanta for that task and for providing

Q-Learning updates to these quantums.



## 4.2 QAgent Architecture

The agent responsible for choosing optimal quantums for every TCB uses a reward function to quantify how well the previous quantum helped to achieve the goals of PULSAR. The agent's reward function can be defined as:

$$R = 1 - t_{ready} + \frac{\Delta}{\Delta + 20} + \begin{cases} 0.5 & \text{if } t_{io} \geq 0.3 \\ 0 & \text{else} \end{cases} \quad (1)$$

Where  $t_{ready}$  is the percentage of time the last task spend in the ready queue,  $\Delta$  is the current allocated time quantum for this current state,  $t_{io}$  is the percentage of time the task spent in the I/O phase. This reward aims to minimize time spent in the ready queue, reward larger quantums that require less context swithces, and incentivize reaching the I/O phase before completing the last context.

The agent takes the continous input space of CPU and I/O usage and bucketizes it into 20 unique states. There are 10 states for CPU usage, split across a binary state for if the I/O wait was above 30%. Each of these 20 states store a time quantum that starts at 10ms.

During each update, the agent is given 7 options for what action it can perform on

the quantum stored in the current state’s bucket. These are ShortenSmall (-1ms), ShortenMedium (-2ms), ShortenLarge (-5ms), Keep, LengthenSmall (+1ms), LengthenMedium (+2ms), and LengthenLarge (+5ms).

To optimize the reward function with respect to the actions it can perform on a quantum, the agent implements a tabular Q-Learning algorithm. Similar to the quantum table, the agent creates a Q-Table of all discrete states multiplied by all actions it could take in that state (for 140 total action-state tuples). Using the new reward value derived, the agent updates the current reward for this action-state tuple using the temporal difference update formula:

$$Q(s, a) + = \alpha(R + \gamma * Q_{max}(s', a') - Q(s, a)) \quad (2)$$

Where  $Q(s, a)$  is the Q value of the current state and action in the Q-Table,  $\alpha$  is the learning rate,  $\gamma$  is the importance of maximizing future rewards, and  $Q_{max}(s', a')$  is the highest Q value currently in this state across all actions we can choose. In our case, we use  $\alpha = 0.1$ ,  $\gamma = 0.6$ .

PULSAR also uses an  $\epsilon$ -greedy algorithm [2] to encourage random search of the reward space. Typically, the action we choose will be the one that has the highest Q-value in the Q-table for the current state. There is, however, a slight chance  $\epsilon$  that we will instead choose a random action for the quantum update instead. In our case, we use  $\epsilon = 0.15$ .

## 5 Evaluation

The core evaluation criteria for all stages of PULSAR include the average percentage of time any task spends on the ready queue, the total number of context switches over the entire period of tracing time, and finally the time spent performing a context switch. For all cases, PULSAR is compared against a baseline model that always allocates 10 ms to all tasks.

### 5.1 Evaluation Environment

#### 5.1.1 Simulation

The simulator captures the pure theory of PULSAR without any of the hardware overhead that a true RTOS would experience. This allowed for rapid iteration and ideal cases for evaluation. The system would step through discrete ticks of time between several tasks that

have been registered. Each task is defined by an abstract list of work it must do, categorized as either CPU or I/O work. Each item of work can be a fixed number of ticks, or a random selection between a range of provided ticks. All heuristic information for every task is collected at every step through the simulation, something which is not realistically feasible for a real implementation.

### **5.1.2 Trivial Bare-Metal Workloads**

Moving from the simulator to real hardware, the first step was to construct a general RTOS that could then be modified to accept a QAgent. This was implemented specifically for Stm32F446 Microcontroller ICs, and was tested on Nucleo Devboards. The general implementation uses one of the board's general purpose timers to trigger an interrupt when a context switch must happen. This triggers the context switch trap which first saves register states before calling the Scheduler Engine's switching logic. This provided a simple preemptive RTOS, which then needed I/O bound syscalls to be viable for our needs. As such, this was implemented by adding an I/O Manager to the Scheduler Engine, which was responsible for tracking TCBs that were waiting on a particular I/O. When I/O was requested from within a task, the I/O Manager would call the interrupt driven version of this I/O before putting the TCB in a waiting queue. When that I/O interrupt response would trigger, the I/O manager would catch it and release all tasks that had been waiting on that value. Currently, the I/O Manager supports GPIO read/write requests, sleep requests, as well as UART read and write events.

From here, the QAgent was added as a step during context switches. Now, the Scheduler Engine would select the next task through a Round-Robin style ready queue. This new task's QAgent would then be activated and requested to update the current state of allocated quanta for that particular task. From here, the agent would notify the Scheduler Engine what an appropriate quantum would be for this task, which the Scheduler Engine would use to set the context switch timer.

With the system implementation in place, we first wanted to prove functionality. To achieve this, we created test workload cases with simpler behaviors that could let us easily see if average wait times are being minimized or not. These test workloads included a system with 2 fully CPU intensive tasks, a system with 1 fully CPU intensive and 1 50/50 CPU vs. I/O intensive tasks. In the case of 2 fully CPU intensive tasks, we would expect average wait times to be around 50% with larger quantum allocations to minimize the amount of context switches that happen. For a case with 1 fully CPU intensive and 1 split behavior workload, we would expect less time being allocated to the CPU task and a larger quantum for the mixed behavior. This would let the I/O task reach its I/O phase quicker, in turn rewarding the CPU bound task with more uninterrupted work time.

### 5.1.3 3D Printer Workload Case Study

After proving that closed form workload configurations can be solved by the RTOS, more realistic workloads inspired by what a 3D printer might need were developed. Each of these workloads are created to both be realistic while also capturing a plethora of different task behaviors; including CPU heavy and I/O heavy tasks, as well as mixed tasks that change through I/O or CPU phases as they execute.

Table 1: 3D Printer Workloads

| Task Name       | Behavior Type | Workload Description                                                                                                                                |
|-----------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| G-CODE Parser   | Phased        | Reads a G-CODE buffer over UART, performs string and floating point comparisons over this buffer before dispatching commands to the stepper motors. |
| Thermal Monitor | Phased        | Reads thermistor values from nozzle and bed, controlling a PID loop to attempt maintaining a consistent temperature                                 |
| E-Stop          | I/O           | Waits for emergency endstop to trigger, forcing an emergency stop if necessary                                                                      |
| Status LED      | I/O           | Blinks a heartbeat LED, sleeping for 500ms between toggles                                                                                          |
| Fan Control     | I/O           | Maintains a duty cycle on a GPIO output for fan controls                                                                                            |

## 5.2 Evaluation Results

### 5.2.1 Simulation

In the simulated environment, both systems run for 9000 ticks of the simulation while tracking average system wait time at every tick. In comparing how both systems respond to a single I/O mixed task and a CPU intensive task (Figure 1), it can be seen that the simulated PULSAR agent reaches an average system wait time of 27.59% of the time, whereas the baseline settles at 31.98%. These results served as promising motivation for turning towards a true bare-metal implementation of PULSAR.



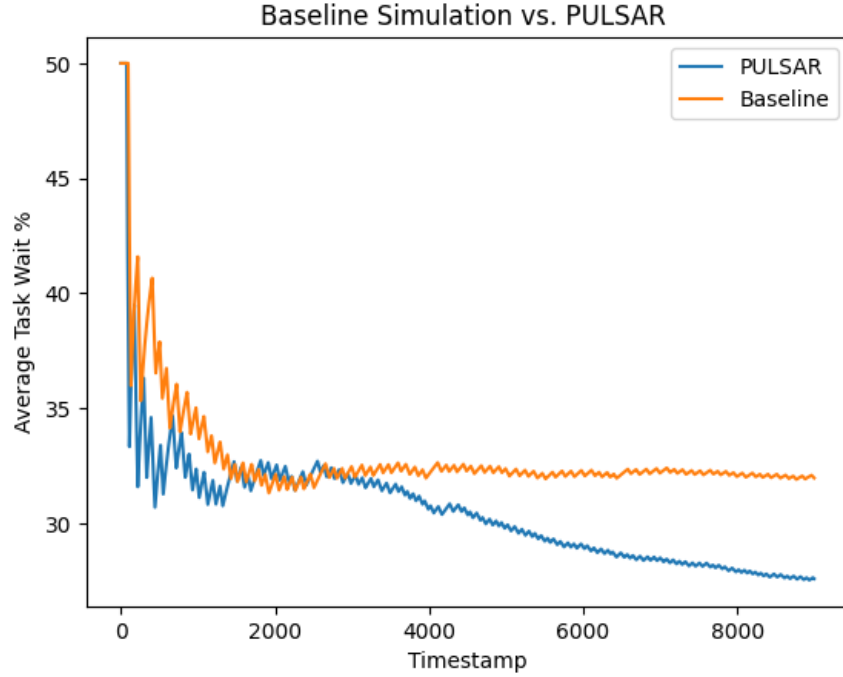


Figure 1: Average task wait percentage for PULSAR versus the baseline in the simulator

### 5.2.2 Trivial Bare-Metal Workload

For the bare-metal comparisons, both systems are given the same collection of tasks and execute for 1000 context switches before reporting their collected heuristics. Furthermore, each test is performed 5 times over with different random seeds to evaluate different choices the Epsilon search may go through. As such, a Kruskal Wallis test will be performed over each distribution versus the baseline distribution to determine if the results from PULSAR can be considered statistically significant. A significance level of 0.05 has been chosen for these tests.

The runs across running 2 CPU heavy tasks found very little change across PULSAR and the baseline. Both systems averaged 50% waiting for both tasks, as makes sense with fully CPU intensive tasks. PULSAR did, however, still achieve a statistically significantly lower distribution with a P value of 0.0053. The difference is, however, from PULSAR's range of 50.210-50.214% to the baseline's 50.218% (Figure 2). PULSAR did further cycle between the range of quantum from 5-15ms. In these dips to a smaller quantum, there were on average more context switches over less time, meaning PULSAR hit 1000 context switches about 25% faster than the baseline.

In testing the 1 CPU, 1 I/O heavy workload configuration, we found PULSAR's average system wait times to be distributed across the range of 23.12-25.65%, while the baseline had very small variation from 27.39% (Figure 3). Kruskal Wallis reported a P-value of 0.009, allowing us to conclude with 95% confidence that the distributions from PULSAR

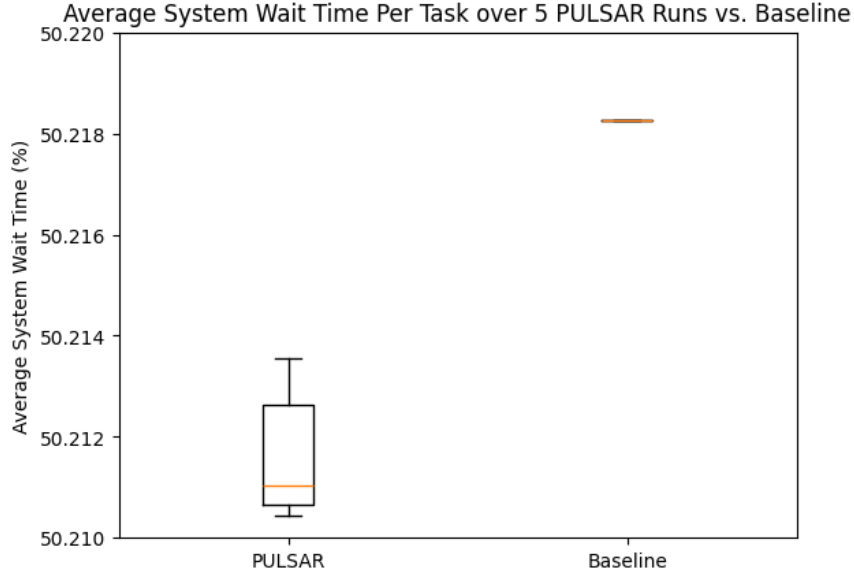


Figure 2: Boxplot distribution of PULSAR runs over 5 different random seeds versus the baseline for the 2 CPU intensive workload

and the baseline are statistically significantly different. PULSAR did, however, allocate less than 10ms for the CPU intensive task at times, leading to more context switches over a smaller amount of time.

### 5.2.3 3D Printer Workload

In running the 3D printer workloads, both baseline and PULSAR average waiting times plummeted compared to the trivial workloads. On average, a given task was only in a waiting queue less than 1% of the time. This is likely due to the fact that in the printer case study, there is no task that is 100% CPU intensive and therefore there are many tasks that can spend most of their time in the I/O waiting state due to minimal CPU activity. The resulting average wait times across the baseline and PULSAR were both minimal and not deemed statistically significant with a P value of 0.43 (Figure 4).

In looking at the allocated quantum behavior across a single task, however, some interesting patterns begin to emerge. For the GCode parser task, the allocation of different quanta behaves like a wave (Figure 5). With the knowledge of the GCode parser task being split between phases of waiting on a UART buffer to fill and performing parsing computations on the resultant buffer, it can be hypothesized that these allocations relate to which of these states the task currently was in: optimizing wait time by providing more time to reach the UART request state while minimizing quanta during CPU heavier phases to allow other tasks time to run.

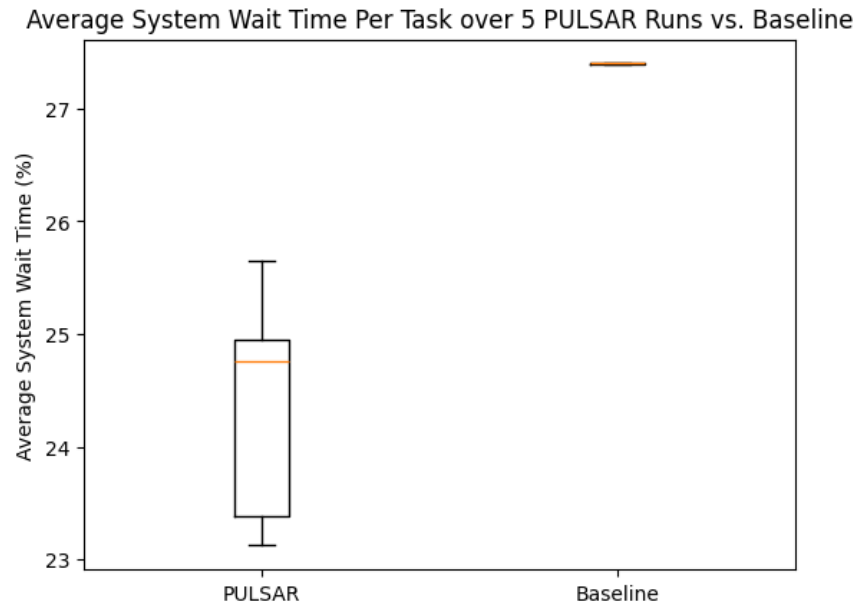


Figure 3: Boxplot distribution of PULSAR runs over 5 different random seeds versus the baseline for the 1 CPU, 1 I/O intensive workload

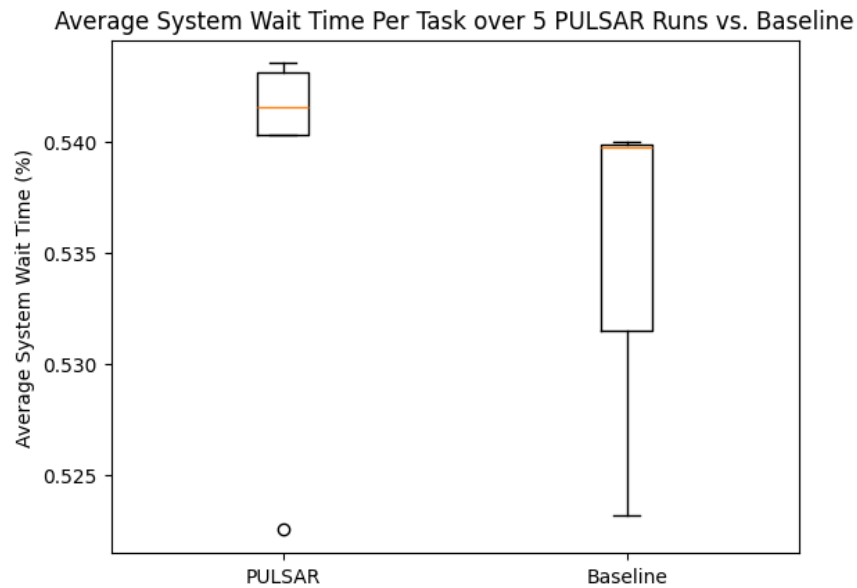


Figure 4: Boxplot distribution of PULSAR runs over 5 different random seeds versus the baseline for the printer workloads

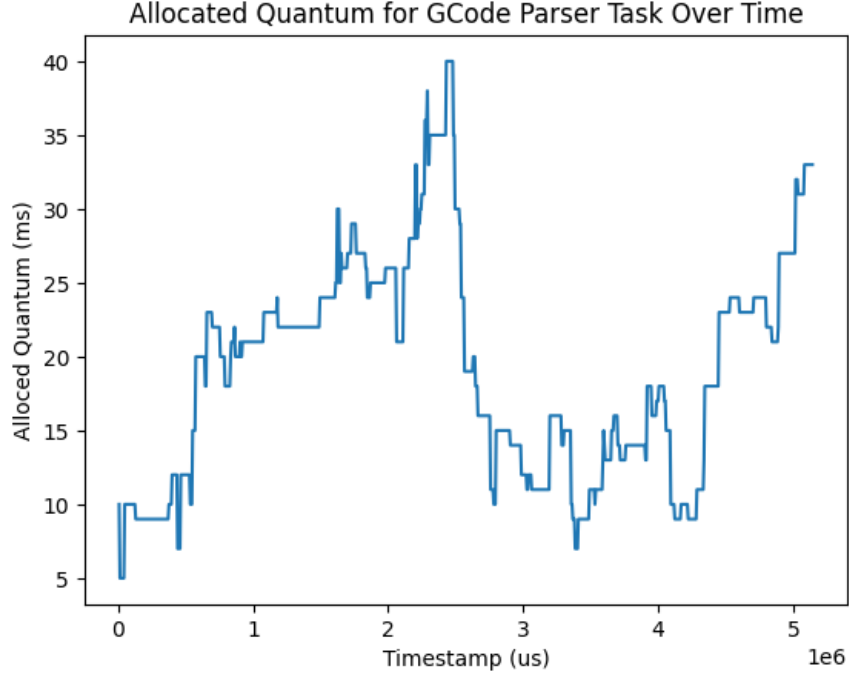


Figure 5: Allocated quanta to the GCode parser task over time

### 5.3 Context Switching CPU and Memory Overhead

The Q-Learning enabled updates taking place in PULSAR do introduce significant overhead in terms of both memory and runtime as compared to the typical baseline. With 4 byte floating points numbers and single byte time slices, the memory overhead required for both the quantum and Q-table of a single TCB is 580 bytes. Quantization to f16 or u16 data types for the Q-values could significantly reduce this overhead, this however has not been tested.

Traces of the context switch function with and without Q-Learning enabled has shown a latency of  $38\mu s$ , with the baseline having a latency of  $5\mu s$ . Further investigation reveals that about 26 of those  $38\mu s$  is spent performing expensive floating point calculations for the Q-table updates. As with the memory footprint, exploration into quantizing these calculations into integer computations could drastically reduce the runtime overhead that comes with PULSAR.

## 6 Conclusions

Across all tests of PULSAR thus far, it has proven to be better if not the same as an existing baseline model. In the case of the 3D Printer tasks specifically, the high I/O nature of all

tasks registered made baseline wait times extremely low to begin with. As seen in figure 5, however, it can be inferred that PULSAR still was able to derive behavior and state change of a phased task over time. This speaks to PULSAR’s ability to converge to unique optimal quantum at different phases of a task’s time. For the GCode parser, PULSAR allocated higher quantum to the I/O entering phase before decreasing again as it transitioned into the CPU-heavy parsing phase. This is the highest value aspect of PULSAR, as it can cheaply approximate task behavior and adapt the unique quantum to this specific behavior.

As for more trivial cases, PULSAR was also able to maintain reduced average wait percentages when compared to the baseline. It did, however, almost always allocate less than 10 ms at different stages of the task’s behavior, leading to more context switching overhead over a shorter period of time. This coupled with the higher computational cost of performing a PULSAR update brings up the tradeoff that PULSAR presents: system waits can be optimized, but at the cost of a more performative operating system that takes up more time itself. Further testing on quantization could bring PULSAR closer to the performance of a traditional static quantum, but the tradeoff will almost certainly always exist to some degree.

## **7 Future Work and Considerations**

### **7.1 Epsilon Decay**

A common practice when using  $\epsilon$ -greedy search is the slowly decay  $\epsilon$  as time goes on. This encourages early randomized search before slowly becoming more stagnant as the system matures. PULSAR currently does not implement  $\epsilon$  decay, but a simple update of  $\epsilon = 0.99\epsilon$  could help to stabilize PULSAR as it finishes early training.

### **7.2 Evaluation Against Prior Art**

PULSAR currently is only tested against a baseline that does not attempt any dynamic resource management, using a comparison against the Q-Learning system that Li et al. [5] use for dynamic prioritization could give more insight to where PULSAR’s strengthes and weaknesses are.

### 7.3 Q Learning Quantization

As mentioned in section 5.4, the significant memory and runtime overhead that PULSAR brings forward could be mitigated through quantization of floating point values. Turning all reward values into 2 byte signed integers and converting all floating point math to this could bring down the runtime cost of evaluation, and would cut memory footprint in half.

### References

- [1] J. W. S. Liu, Real-Time Systems (Prentice Hall, Englewood Cliffs, N.J., 2000).
- [2] Sutton, Richard S., and Andrew G. Barto. *Reinforcement learning: An introduction*. Vol. 1. No. 1. Cambridge: MIT press, 1998.
- [3] C. L. Liu and J. W. Layland, Scheduling algorithms for multiprogramming in a hard-real-time environment, J. ACM, vol. 20, no. 1, pp. 46–61, 1973.
- [4] Sun, Xiaoxuan, et al. "Dynamic operating system scheduling using double DQN: A reinforcement learning approach to task optimization." *2025 8th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*. IEEE, 2025.
- [5] Li, Pochun, et al. "Reinforcement learning for adaptive resource scheduling in complex system environments." *2024 5th International Symposium on Computer Engineering and Intelligent Communications (ISCEIC)*. IEEE, 2024.
- [6] Watkins, C.J.C.H., Dayan, P. Q-learning. Mach Learn 8, 279–292 (1992). <https://doi.org/10.1007/BF00992698>
- [7] Zhong, W. "A Multi-Level Feedback Queue Optimization Method Based on Reinforcement Learning and Dynamic Time Slice Scheduling." *Automation and Machine Learning*, Clausius Scientific Press, vol. 6, no. 1, pp. 111-117 (2025).